TradeOps

Milestone 2: Problem Definition, Extended ERD, Relational Model, and PostgreSQL Implementation

Item	Details
Course	IE6750 Data Warehousing & Integration, Northeastern University
Assignment	Milestone 2: Extended ERD, Relational Model, PostgreSQL Implementation
Group Number	Group 08
Member	Jesus D. Jimenez Ballestas
Project Title	TradeOps: A Data Warehouse for Prop-Firm Futures Trading Operations
Data Source	Generated through a seeded Python script (synthetic), using real contract specifications
Database	PostgreSQL 16, database tradeops, schema tradeops
Repository	https://github.com/Jesus-D-Jimenez/tradeops
Date	October 2026

1. Project Summary

This project focuses on building a data warehouse for a futures trading operation that uses capital provided by proprietary trading firms, or "prop firms," instead of trading only with personal funds. The operation manages dozens of evaluation and funded accounts across several firms, while a single trade can be copied to every active account at the same time. Because each firm has different rules, fees, and payout schedules, the information needed to evaluate overall performance is spread across multiple dashboards and platform exports. The main goal of the project is to bring this information together in one place so it becomes easier to answer an important question: after all fees are included, is the operation actually profitable, and which strategies, instruments, and firms contribute the most to the results?

2. Operational Business Context

A prop firm gives traders access to an evaluation account for a fee. For example, a trader might pay a monthly fee for a $50,000 evaluation account. To pass the evaluation, the trader must reach a profit target while keeping the account balance above a trailing drawdown limit that moves with the account's highest balance. Some firms also have daily loss limits and consistency rules that restrict how much of the total profit can come from one trading day. After passing the evaluation, the trader receives a funded account, usually after paying an activation fee. The trader can then request payouts and typically keeps most of the profit, commonly around 90%. If the account balance reaches the drawdown limit, the account is lost.

This project looks at that process on a larger scale. The operational database needs to support the following day-to-day activities:

- **Account management:** purchasing evaluation accounts, paying monthly and reset fees, and activating funded accounts after an evaluation is passed.
- **Trade execution:** a trade is placed in a leader account, and copy-trading software sends the same trade to each follower account, although position sizes and fills can differ.
- **End-of-day statements:** each firm reports information such as the account's ending balance, high-water mark, and drawdown floor.
- **Rule monitoring:** tracking events such as drawdown breaches, daily loss-limit violations, and consistency-rule flags.

- **Payouts:** recording withdrawal requests from funded accounts based on each firm's payout schedule.

These are transactional processes because they usually add or update a small number of records at a time and mainly depend on current data. This matches the OLTP workload described in Section 3.1 of Vaisman and Zimányi (2022). The type of analysis needed for this project is different because it requires combining and summarizing historical information, which supports the need for a separate data warehouse.

3. Problem Definition

Problem statement. The operation does not currently have an easy way to measure its true return because expenses, such as evaluation, reset, and activation fees, are stored separately from trading results and payout income. Each prop firm also reports its information in a different format. As a result, it is difficult to determine which strategies, trading sessions, or instruments are connected to the drawdown violations that cause accounts to fail. It is also difficult to compare copied trades with the original leader trade. This project first creates an operational database that records these events, which is the focus of this milestone, and will later use a data warehouse to combine the information for multidimensional analysis.

The project is designed to answer the following business questions:

1. What is the operation's net return, calculated as payouts received minus all fees, by firm, plan, and month?
2. Which strategies and instruments generate the most net P&L, and which ones are associated with the most rule violations?
3. How does trading performance differ by session and on major economic event days such as CPI, NFP, and FOMC?
4. What percentage of evaluations are passed, how long does it take to pass them, and how long do funded accounts remain active?
5. How much additional slippage is created when trades are copied from the leader account?

Scope. The project includes futures trades, account lifecycles, fees, balances, rule violations, and payouts from January 2025 through August 2026. Live market data, order-book data, and personal tax reporting are outside the scope of the project.

Milestone 2 Additions

Milestone 1 defined the problem and created the first version of the operational database. This milestone builds on that work by adding the extended ERD (Section 4), the mapping to a normalized relational model (Section 5), the PostgreSQL implementation (Section 6), the data population methodology (Section 7), and a first list of candidate dimensions, hierarchies, and measures for the warehouse (Section 8). Compared with Milestone 1, the main changes to the database are:

- **Platform:** the schema now runs natively in PostgreSQL 16, while the Milestone 1 version was tested in DuckDB and MySQL.
- **Constraints:** every constraint now has a name, and 19 new CHECK rules were added, for a total of 30. These rules enforce business logic such as the Evaluation and Funded plan rules, closing dates for closed accounts, paid dates for paid payouts, and exit times that come after entry times.
- **New relationship:** Product.FullSizeRoot links each micro contract to its full-size product, for example MNQ to NQ. Copied trades often use the micro version of the leader's contract, so this link is needed to compare the two.

- **Indexes:** 15 secondary indexes on the foreign keys and on Trade.EntryTime support the joins and date filters that the ETL process will run in later milestones.

4. Extended Entity-Relationship Model

The extended ERD was created in draw.io using crow's foot notation and is shown in F 1 (the editable file is TradeOps_Diagrams.drawio). The model contains 18 entity types: 16 entities that become tables in the database and two subtypes of AccountPlan. It also uses four features of the extended ER model, which are described below:

- **Specialization:** AccountPlan is specialized into EvaluationPlan and FundedPlan based on the PlanType attribute. The specialization is disjoint, because a plan can only be one of the two types, and total, because every plan must belong to one of them. Evaluation plans have a ProfitTarget and MinTradingDays, while funded plans have a ProfitSplitPct.
- **Weak entities:** Fill depends on Trade and is identified by the composite key (TradeID, LegNo), which means a fill cannot exist without the trade it belongs to. In the same way, DailyAccountBalance depends on Account and is identified by (AccountID, TradeDate).
- **Recursive relationships:** the model includes three relationships between an entity and itself. An evaluation Account earns a funded Account (earns), a leader Trade is copied into follower Trades (copies), and a micro Product is the smaller version of a full-size Product (micro of).
- **Associative entity:** StrategyApproval resolves the many-to-many relationship between Strategy and Product. It is modeled as its own entity because it carries attributes of its own, MaxContracts and EnabledDate.

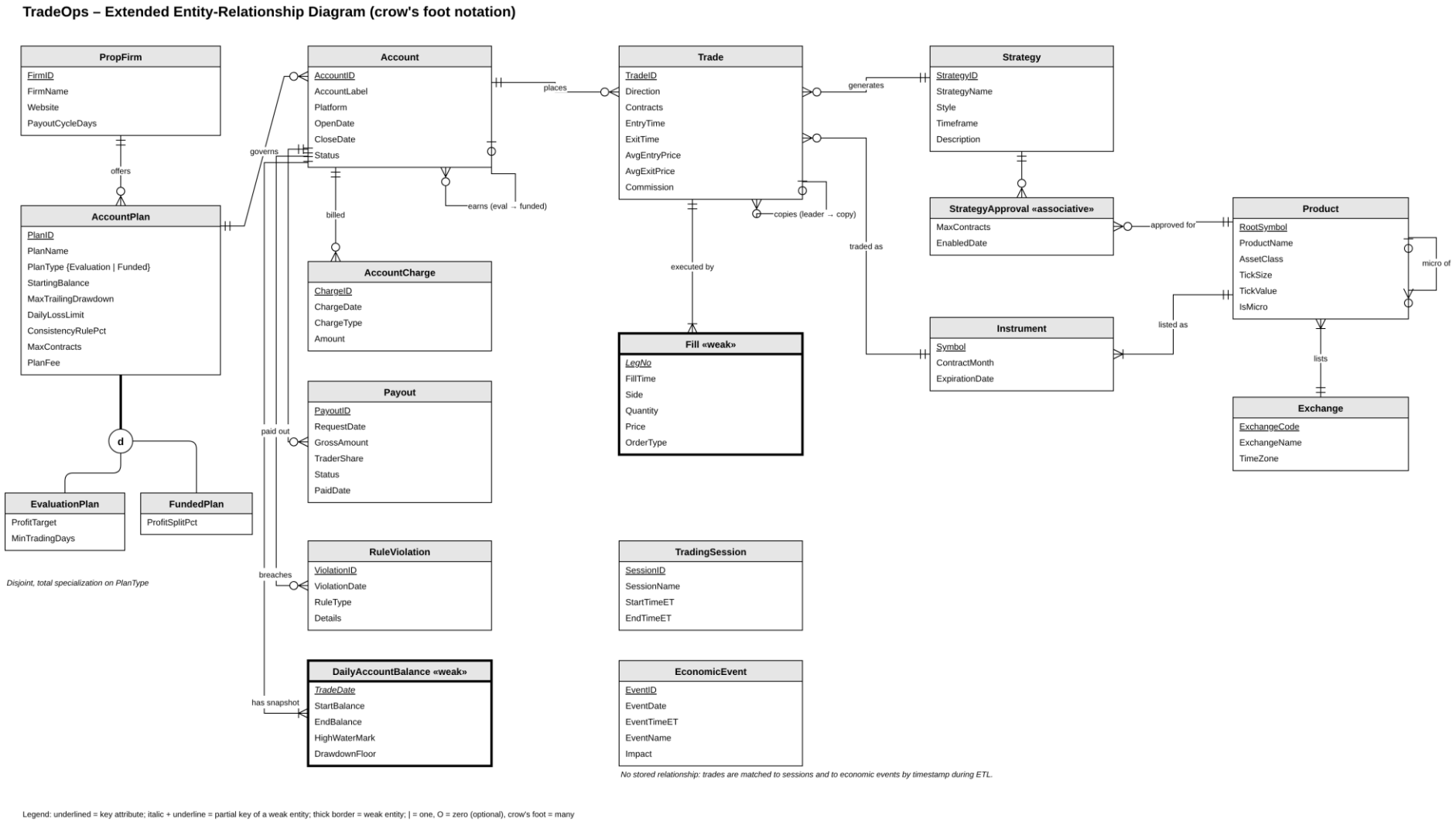
TradingSession and EconomicEvent do not have stored relationships with the other entities. Instead, the session of a trade and any nearby economic release can be found by comparing timestamps, and the ETL process will perform this lookup when it builds the warehouse. Storing a SessionID on every trade would repeat information that the entry time already determines.

Relationships and Cardinalities

Relationship	Parent (one side)	Child (many side)	Cardinality	Notes
offers	PropFirm	AccountPlan	1 to 0..*	Each plan belongs to exactly one firm
governs	AccountPlan	Account	1 to 0..*	Plan rules apply to every account bought on that plan
earns	Account (evaluation)	Account (funded)	0..1 to 0..*	Recursive; ParentAccountID
billed	Account	AccountCharge	1 to 0..*	Evaluation, activation, reset, and monthly fees
paid out	Account	Payout	1 to 0..*	Only funded accounts receive payouts
breaches	Account	RuleViolation	1 to 0..*	Drawdown, daily loss, consistency, contract limit
has snapshot	Account	DailyAccountBalance	1 to 1..*	Weak entity; one row per account per trading day
places	Account	Trade	1 to 0..*	
executed by	Trade	Fill	1 to 1..*	Weak entity; entry and exit legs
copies	Trade (leader)	Trade (copy)	0..1 to 0..*	Recursive; LeaderTradeID is NULL for leader trades
generates	Strategy	Trade	1 to 0..*	
traded as	Instrument	Trade	1 to 0..*	A trade is in one contract month (e.g., NQZ5)

listed as	Product	Instrument	1 to 1..*	Quarterly or monthly contract months
lists	Exchange	Product	1 to 1..*	CME, CBOT, NYMEX, COMEX
approved for	Strategy / Product	StrategyApproval	M:N	Associative entity
micro of	Product (full-size)	Product (micro)	0..1 to 0..*	Recursive; FullSizeRoot

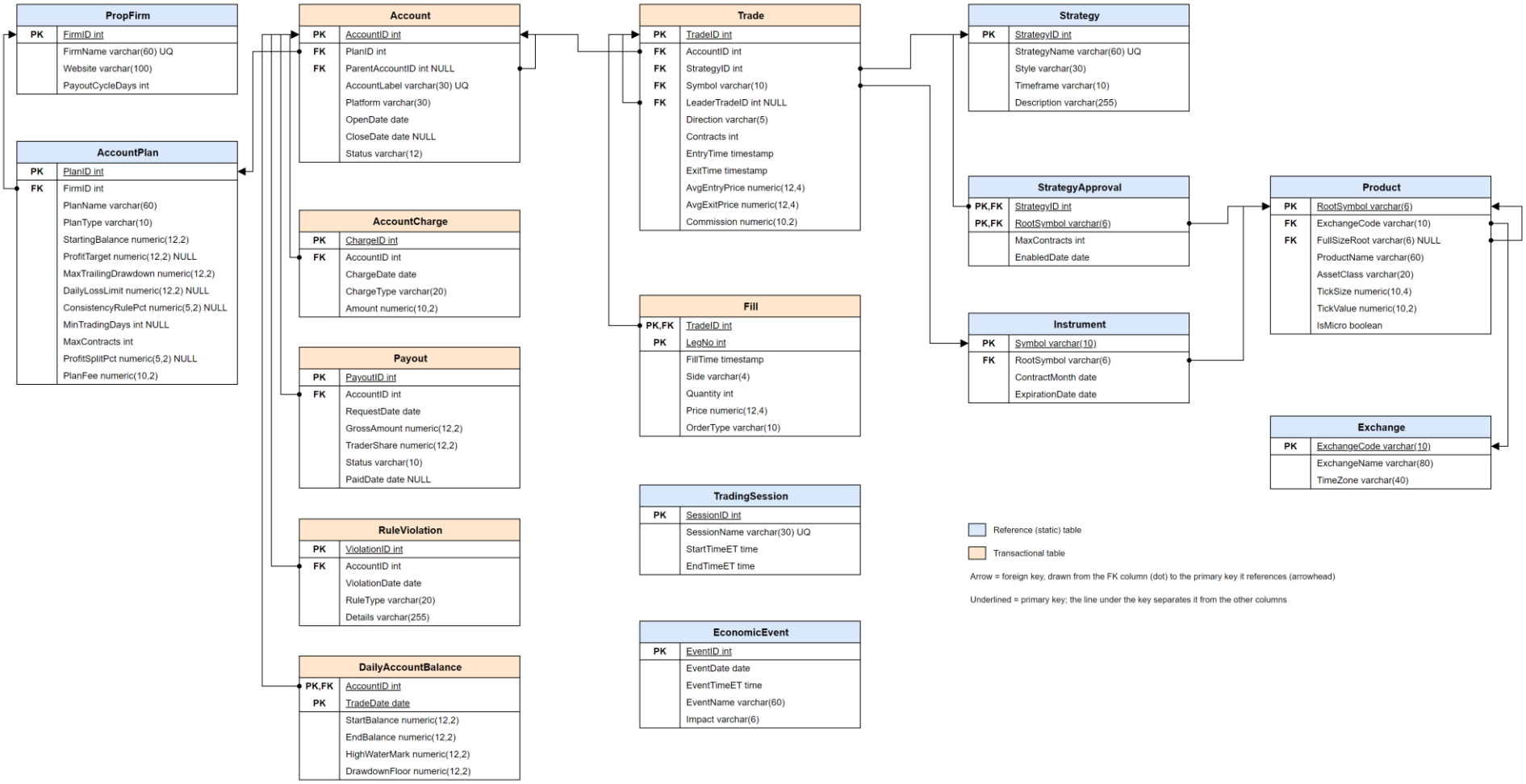
F 1. Extended Entity-Relationship Diagram



F 1. TradeOps EER diagram

F 2. Relational Model

TradeOps – Relational Model (PostgreSQL schema tradeops, 16 tables)



PK = primary key, FK = foreign key, UQ = unique, NULL = optional column. All other columns are NOT NULL. CHECK constraints and indexes are listed in sql/01_schema.sql.

F 2. TradeOps relational model as implemented in PostgreSQL. Each line is a foreign key.

5. Mapping the ERD to the Relational Model

The ERD was mapped to relations using the standard ER-to-relational mapping rules. T 1 summarizes how each type of construct was handled, and F 2 shows the resulting relational model.

EER construct	Mapping rule used	Result in TradeOps
Strong entity	One relation; the key attribute becomes the primary key	PropFirm, Account, Trade, Strategy, Product, and the others
1:N relationship	Foreign key on the many side	Account.PlanID, Trade.AccountID, Trade.StrategyID, Instrument.RootSymbol, ...
Recursive 1:N	Nullable foreign key to the same table	Account.ParentAccountID, Trade.LeaderTradeID, Product.FullSizeRoot
Weak entity	Primary key = owner's key + partial key; FK with ON DELETE CASCADE	Fill (TradeID, LegNo); DailyAccountBalance (AccountID, TradeDate)
M:N with attributes	Separate relation keyed by both foreign keys	StrategyApproval (StrategyID, RootSymbol)
Disjoint, total specialization	Single relation with a type attribute; subtype columns are nullable, and CHECK rules enforce which subtype uses them	AccountPlan with PlanType; ck_accountplan_eval and ck_accountplan_funded

T 1. ER-to-relational mapping rules applied

For the AccountPlan specialization, a single table was chosen instead of separate EvaluationPlan and FundedPlan tables. The two subtypes share eight of their eleven attributes and differ in only three, and separate tables would also require Account to have two different foreign keys. Two CHECK constraints make sure the single table still follows the subtype rules: an evaluation plan must have a profit target and no profit split, while a funded plan must have a profit split and no profit target.

Relational Schema

T 2 lists each relation with its primary key, its other attributes, and the table that each foreign key references.

Relation	Primary key	Other attributes	Foreign keys
PropFirm	FirmID	FirmName, Website, PayoutCycleDays	-
AccountPlan	PlanID	FirmID, PlanName, PlanType, StartingBalance, ProfitTarget, MaxTrailingDrawdown, DailyLossLimit, ConsistencyRulePct, MinTradingDays, MaxContracts, ProfitSplitPct, PlanFee	FirmID → PropFirm
Exchange	ExchangeCode	ExchangeName, TimeZone	-
Product	RootSymbol	ProductName, ExchangeCode, FullSizeRoot, AssetClass, TickSize, TickValue, IsMicro	ExchangeCode → Exchange; FullSizeRoot → Product
Instrument	Symbol	RootSymbol, ContractMonth, ExpirationDate	RootSymbol → Product
Strategy	StrategyID	StrategyName, Style, Timeframe, Description	-
StrategyApproval	StrategyID, RootSymbol	MaxContracts, EnabledDate	StrategyID → Strategy; RootSymbol → Product
TradingSession	SessionID	SessionName, StartTimeET, EndTimeET	-
EconomicEvent	EventID	EventDate, EventTimeET, EventName, Impact	-
Account	AccountID	PlanID, ParentAccountID, AccountLabel, Platform, OpenDate, CloseDate, Status	PlanID → AccountPlan; ParentAccountID → Account
AccountCharge	ChargeID	AccountID, ChargeDate, ChargeType, Amount	AccountID → Account
Trade	TradeID	AccountID, StrategyID, Symbol, LeaderTradeID, Direction, Contracts, EntryTime, ExitTime, AvgEntryPrice, AvgExitPrice, Commission	AccountID → Account; StrategyID → Strategy;

			Symbol → Instrument; LeaderTradeID → Trade
Fill	TradeID, LegNo	FillTime, Side, Quantity, Price, OrderType	TradeID → Trade
DailyAccountBalance	AccountID, TradeDate	StartBalance, EndBalance, HighWaterMark, DrawdownFloor	AccountID → Account
RuleViolation	ViolationID	AccountID, ViolationDate, RuleType, Details	AccountID → Account
Payout	PayoutID	AccountID, RequestDate, GrossAmount, TraderShare, Status, PaidDate	AccountID → Account

T 2. Relational schema (16 relations)

Normalization

Each relation was checked against the first three normal forms by reviewing its functional dependencies.

- **First normal form:** all attributes are atomic. For example, the executions of a trade are stored as separate Fill rows instead of a list of prices, and each strategy-product approval is stored as its own StrategyApproval row instead of a list of symbols on Strategy.
- **Second normal form:** the only composite keys are in Fill, DailyAccountBalance, and StrategyApproval, and each non-key attribute in those tables depends on the whole key. For example, the Price of a fill depends on both the trade and the leg, and EndBalance depends on both the account and the date.
- **Third normal form:** no non-key attribute depends on another non-key attribute. The decompositions shown in T 3 removed the transitive dependencies that a single flat trading journal would contain.

Transitive dependency in a flat journal	**Decomposition**
AccountID → PlanID → ProfitTarget, MaxTrailingDrawdown, ProfitSplitPct, PlanFee	Plan rules moved to AccountPlan; Account keeps only PlanID
PlanID → FirmID → FirmName, PayoutCycleDays	Firm details moved to PropFirm
Symbol → RootSymbol → TickSize, TickValue, ExchangeCode	Contract specifications moved to Product; Instrument keeps contract-month data only
RootSymbol → ExchangeCode → ExchangeName, TimeZone	Exchange details moved to Exchange
TradeID → StrategyID → StrategyName, Style, Timeframe	Strategy details moved to Strategy

T 3. Decompositions that bring the schema to 3NF

Two attributes are intentionally redundant and are documented as controlled denormalization. Trade.AvgEntryPrice and Trade.AvgExitPrice could be calculated from the Fill rows, but trading platforms report them on the trade itself, and storing them avoids aggregating fills every time a trade is displayed. The integrity check in Section 6.3 confirms that the fills of every trade net to a flat position. Net P&L, on the other hand, is not stored at all. Instead, it is calculated in the view v_trade_pnl from prices, tick size, and tick value, so it cannot disagree with the underlying data.

6. PostgreSQL Implementation

The relational model was implemented in PostgreSQL 16 in a dedicated schema named tradeops. The build is divided into four scripts, which makes it possible to rebuild the database from scratch in under a minute:

Script	**Purpose**
sql/01_schema.sql	Drops and recreates the tradeops schema: 16 tables, all constraints, and 15 indexes
sql/02_load.sql	Loads the 16 CSV files with \copy in dependency order inside one transaction, then sets FullSizeRoot for micro products

sql/03_views.sql	Creates v_trade_pnl, which calculates P&L in ticks and dollars, net of commission, and holding time
sql/04_checks.sql	Prints row counts and runs integrity checks

The database can be rebuilt from the project folder with the following commands:

```
createdb tradeops
psql -d tradeops -f sql/01_schema.sql
psql -d tradeops -f sql/02_load.sql
psql -d tradeops -f sql/03_views.sql
psql -d tradeops -f sql/04_checks.sql
```

6.1 Constraints

Constraint type	Count	Examples
Primary key	16	pk_trade (TradeID); pk_fill (TradeID, LegNo)
Foreign key	17	fk_trade_leader (LeaderTradeID → Trade); fk_fill_trade with ON DELETE CASCADE
Unique	7	uq_account_label; uq_instrument_root_month (RootSymbol, ContractMonth)
Check	30	ck_trade_times (ExitTime ≥ EntryTime); ck_payout_paid (Paid status ⇔ PaidDate present); ck_account_closed (Active ⇔ no CloseDate); ck_accountplan_eval / ck_accountplan_funded
Index	15	ix_trade_entrytime; one index on every foreign key column

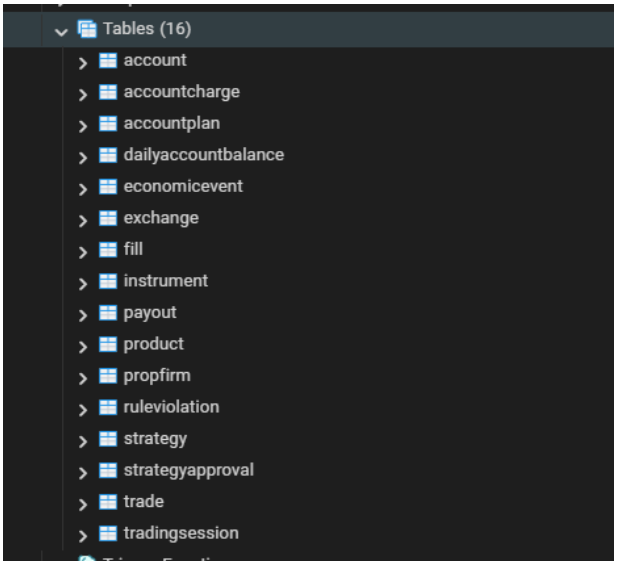
T 4. Constraints and indexes in the tradeops schema

6.2 Loaded Row Counts

Table	Type	Rows
PropFirm	Reference	3
AccountPlan	Reference	8
Exchange	Reference	4
Product	Reference	12
Instrument	Reference	164
Strategy	Reference	5
StrategyApproval	Reference	24
TradingSession	Reference	5
EconomicEvent	Reference	245
Account	Transactional	134
AccountCharge	Transactional	220
Trade	Transactional	6,322
Fill	Transactional	13,944
DailyAccountBalance	Transactional	2,916
RuleViolation	Transactional	143
Payout	Transactional	63

T 5. Rows loaded per table (24,228 total)

F 3 shows the result in pgAdmin 4. The left side lists the 16 tables in the tradeops schema, and the right side shows the number of rows that PostgreSQL 16 returned for each table, which matches T 5.



	table_name text	row_count bigint
1	PropFirm	3
2	AccountPlan	8
3	Exchange	4
4	Product	12
5	Instrument	164
6	Strategy	5
7	StrategyApproval	24
8	TradingSession	5
9	EconomicEvent	245
10	Account	134
11	AccountCharge	220
12	Trade	6322
13	Fill	13944
14	DailyAccountBalan...	2916
15	RuleViolation	143
16	Payout	63

F 3. Schema tradeops and rows loaded per table

6.3 Integrity Checks

All 24,228 rows were loaded with every constraint enforced and without any errors. The checks below go beyond what constraints can express and confirm that the data is internally consistent. The results come from the output of sql/04_checks.sql:

Check	Result
Trades whose fills do not net to a flat position	0
Copied trades in a different direction or contract family than their leader	0
Funded accounts whose parent account is not an evaluation	0
Trades on a product the strategy is not approved for	0
Micro products without a full-size parent	0
Trade date range	2025-01-03 to 2026-08-31 (420 trading days)
Leader trades / copied trades	1,014 / 5,308

T 6. Integrity check results

F 4 shows the first trades in the view v_trade_pnl. For example, trade 6 is a full-size YM copy of the micro MYM leader trade 5, and it closed one tick worse than the leader (−51 vs. −50 ticks). This difference is the copy-trading slippage that the warehouse will measure.

	tradeid integer	accountid integer	symbol character varying (10)	traderole text	direction character varying (5)	contracts integer	pnlticks numeric	grosspnl numeric	commission numeric (10,2)	netpnl numeric	holdingseconds integer
1	1	1001	MNQH5	Leader	Short	6	-89.00	-267.00	6.00	-273.00	2144
2	2	1001	MGCG5	Leader	Long	5	-60.00	-300.00	5.00	-305.00	2041
3	3	1001	MCLG5	Leader	Short	10	51.00	510.00	10.00	500.00	4757
4	4	1001	MESH5	Leader	Short	10	-19.00	-237.50	10.00	-247.50	1323
5	5	1001	MYMH5	Leader	Short	10	-50.00	-250.00	10.00	-260.00	5606
6	6	1002	YMH5	Copy	Short	2	-51.00	-510.00	8.00	-518.00	5618
7	7	1001	M2KH5	Leader	Short	10	48.00	240.00	10.00	230.00	1192
8	8	1002	RTYH5	Copy	Short	2	47.00	470.00	8.00	462.00	1163
9	9	1001	MESH5	Leader	Long	10	-25.00	-312.50	10.00	-322.50	1011
10	10	1003	MESH5	Copy	Long	10	-25.00	-312.50	10.00	-322.50	972
11	11	1001	MGCG5	Leader	Long	5	-60.00	-300.00	5.00	-305.00	5092
12	12	1003	MGCG5	Copy	Long	5	-61.00	-305.00	5.00	-310.00	5078
13	13	1001	MCLG5	Leader	Short	10	-34.00	-340.00	10.00	-350.00	1101
14	14	1002	CLG5	Copy	Short	2	-34.00	-680.00	8.00	-688.00	1116
15	15	1003	MCLG5	Copy	Short	10	-34.00	-340.00	10.00	-350.00	1109

F 4. Sample rows in PostgreSQL

7. Data Population Methodology

Real account statements from prop firms are private, and each firm reports them in a different format. For this reason, the data was generated with a seeded Python script (generate_data.py). Because the seed is fixed (6750), the same data is produced every time the script is run. As in Milestone 1, the design is based on a real trading operation I run, but the records themselves are synthetic and do not include my actual trades. To keep the data realistic, the reference data is based on real-world sources, and the transactions come from a simulation that follows the rules of each firm.

Static Reference Data

- **Products and exchanges:** 12 CME Group futures, made up of 6 full-size contracts and their micro versions, with the real tick sizes and tick values published by CME. The products are spread across 4 exchanges and 3 asset classes: equity index, energy, and metals.
- **Instruments:** 164 contract months were generated from real listing cycles. Equity index futures use quarterly months (H, M, U, Z), crude oil uses monthly contracts, and gold uses even months, and each expiration date follows the rule for its product.
- **Prop firms and plans:** the data includes 3 firms (Topstep, Lucid Trading, and MyFundedFutures) and 8 evaluation and funded plans. The prop-firm names are real, but the fees, profit targets, drawdowns, and profit splits are simplified versions of each firm's published plans.
- **Economic calendar:** 245 events from January 2025 to August 2026 follow common release patterns, such as nonfarm payrolls on the first Friday of the month, CPI in the middle of the month, and FOMC decisions in the eight scheduled meeting months. The calendar also includes weekly jobless claims and EIA crude inventory reports.
- **Strategies and sessions:** the data includes 5 strategies with different styles and timeframes, 24 strategy-product approvals, and 5 trading sessions (Asia, London, NY Open, NY Midday, and NY Close).

Transactional Data

The simulation steps through every business day from January 2025 to August 2026 and applies the same rules that the firms use:

- Prices for each product follow a random walk with their own volatility and drift and are rounded to the tick size of the product.
- Each day, every strategy may produce a signal at its own time of day. For example, Opening Range Breakout produces its signal after 9:30 ET. The News Fade strategy only trades on days with a high-impact 8:30 ET release from the economic calendar, which connects the transactional data to the reference data.
- The oldest active account places the leader trade, and the copy-trading step repeats it in the other active accounts. About 8% of copies are skipped, which reflects a copier being paused. The 50K accounts trade the micro contract, while the 150K accounts trade the full-size contract, and position size is based on a fixed dollar risk per trade. Copies are filled up to 3 seconds later and up to one tick worse than the leader, which creates slippage that can be measured.
- Every trade has one entry fill and one or two exit fills, because winning trades are sometimes scaled out in two legs. This results in an average of 2.2 fills per trade, using market, limit, and stop order types.
- At the end of each day, the simulation records a balance snapshot and updates the high-water mark and trailing drawdown floor. A daily loss limit hit or a drawdown breach is recorded as a RuleViolation. A breached evaluation is either reset, which adds a reset fee, or marked Failed, while a breached funded account is marked Breached.
- An evaluation that reaches its profit target, meets the minimum number of trading days, and passes the consistency rule is marked Passed and opens a funded account, which is linked through ParentAccountID. If a single day makes up too much of the total profit, a Consistency violation is recorded and the account continues trading.
- Funded accounts request a payout once they are at least $1,000 in profit and the payout cycle of the firm has passed. The trader receives the profit split of the plan, and a small share of the requests are denied.
- Fees follow the billing of each firm: an evaluation fee when an account is opened, a monthly renewal every 30 days, reset fees, and an activation fee for funded plans that charge one.

As a result, the dataset contains a reasonable volume of transactional data with real variety: 134 accounts (102 evaluations and 32 funded accounts) that ended as Passed, Failed, Breached, or still Active; 6,322 trades across 107 contract months; 13,944 fills; 2,916 daily snapshots; 220 fee charges; 143 rule violations; and 63 payout requests, of which 59 were paid and 4 were denied.

Loading Process

The generator writes one CSV file for each table. The 02_load.sql script then loads these files with the PostgreSQL \copy command in dependency order, starting with the reference tables, then Account, and finally the tables that reference Account and Trade. The load runs inside a single transaction, which means a single bad row rolls back the entire load. Because every constraint is active during the load, PostgreSQL itself validates the generated data. Finally, the ANALYZE command updates the planner statistics.

8. Potential Dimensions, Hierarchies, and Measures

The operational schema was designed with the future warehouse in mind. This section presents a first draft of the multidimensional model, which may be adjusted in later milestones as the course progresses.

Candidate Facts

Candidate fact	Grain	Source tables
Trade (transaction fact)	One row per closed trade	Trade, Fill, v_trade_pnl
Daily Account Balance (periodic snapshot)	One row per account per trading day	DailyAccountBalance
Account Cash Flow	One row per fee charge or payout	AccountCharge, Payout
Account Lifecycle (accumulating snapshot)	One row per account, updated as it passes, breaches, or closes	Account, RuleViolation

T 7. Candidate fact tables

Dimensions and Hierarchies

Dimension	Source	Hierarchy (leaf → root)	Change type
Date	Generated	Date → Month → Quarter → Year; Date → Week	Static
Time of Day / Session	TradingSession	Minute → Hour → Session	Static
Account	Account, AccountPlan, PropFirm	Account → Plan → Firm; Account → Plan Type (Evaluation, Funded)	Status changes often (Active → Passed/Breached): SCD Type 2
Instrument	Instrument, Product, Exchange	Contract Month → Product → Full-size Family → Asset Class; Product → Exchange	New contracts every month or quarter (rapidly growing)
Strategy	Strategy, StrategyApproval	Strategy → Style	Rule changes: SCD Type 2
Economic Event	EconomicEvent	Event → Event Type → Impact	Static once released
Trade Role	Trade.LeaderTradeID	Leader / Copy	Static
Platform	Account.Platform	Platform (TopstepX, Tradovate)	Slowly changing

T 8. Candidate dimensions and hierarchies

The Instrument hierarchy is especially useful for this project. Contract months roll up into products, and the new FullSizeRoot link allows MNQ and NQ to roll up into a single Nasdaq-100 family. As a result, the leader trade, which often uses NQ, can be compared with its copies, which often use MNQ, within the same group. The Account dimension also contains a recursive hierarchy (evaluation → funded account) that can be used to follow an account through its entire lifecycle.

Measures and Additivity

As discussed in Section 3.1.2 of Vaisman and Zimányi (2022), not every measure should be aggregated in the same way. Using the flow, level, and unit classification from Module 4, each measure in T 9 has its own set of valid aggregation functions:

Measure	Fact	Type	Valid aggregation
Net P&L, Gross P&L	Trade	Flow	SUM along all dimensions
Commission	Trade	Flow	SUM
Contracts traded, Trade count	Trade	Flow	SUM / COUNT
P&L in ticks	Trade	Flow	SUM, AVG
Slippage vs. leader (ticks)	Trade	Flow	SUM, AVG
Holding time (seconds)	Trade	Unit	AVG, MIN, MAX (not SUM)
Entry / exit price	Trade	Unit	AVG, MIN, MAX (not SUM)

End balance, High-water mark, Drawdown floor	Daily Balance	Level	SUM across accounts; AVG/MIN/MAX or last value over time
Drawdown cushion (EndBalance – Floor)	Daily Balance	Level	Same as above; shows risk of breach
Fee amount, Payout amount (trader share)	Cash Flow	Flow	SUM
Net return (payouts – fees)	Cash Flow	Flow (derived)	SUM
Pass rate, Win rate	Lifecycle / Trade	Unit (ratio)	Recompute from counts; never SUM
Days to pass, Funded survival days	Lifecycle	Flow	AVG, MIN, MAX

T 9. Candidate measures with their additivity

Thinking Ahead

The assignment also asks three questions about whether the data is ready for the warehouse design in later milestones. Each one is answered below:

- **Slowly and rapidly changing data:** Yes. Prop firms can change plan rules, fees, and payout cycles over time, and strategy approvals can also change. These attributes change slowly, so they are good candidates for Type 2 slowly changing dimensions that keep historical versions, because results must be judged against the rules that were in force when each trade was placed. Other data changes much faster. An account can move from Active to Passed to Breached within a few weeks, and new contract months are added every month or quarter as contracts roll.
- **Three or more measures:** Yes. The four candidate facts provide more than ten measures. These include additive measures such as P&L, fees, and payouts, semi-additive level measures such as account balances, and non-additive unit measures such as prices, holding time, and ratios.
- **Enough dimensions:** Yes. Trades can be analyzed by date, session, account, plan, firm, strategy, instrument, exchange, economic event, and leader or copy role. Together, these dimensions cover every business question listed in Section 3.

References

Vaisman, A., & Zimányi, E. (2022). *Data warehouse systems: Design and implementation* (2nd ed.). Springer.